Shiv's Data Ledger — Portfolio: Technical Documentation

A personal portfolio site for Shiv Kumar (Shivmahlan), a data-science student who builds ML pipelines, computer-vision tools, and "honest explanations." The site is a single-page application with scroll-driven motion, an interactive profile hub, and live data pulled client-side (LeetCode stats) with graceful fallbacks.

Live URL: https://portfolio.shivmahlan141.workers.dev

---

1. Tech Stack (verified from package.json / source)

Layer	Choice	Version
Build tool	Vite	^7.3.1
UI framework	React	^19.2.0
React renderer	react-dom	^19.2.0
Animation engine	GSAP (+ ScrollTrigger plugin)	^3.15.0
Motion / a11y	Motion (Framer Motion) — useReducedMotion only	^12.34.3
Icons	@phosphor-icons/react	^2.1.10
Styling	Plain CSS + CSS custom properties (no Tailwind)	—
Linting	ESLint 9 + react-hooks + react-refresh plugins	^9.39.1
Language	JavaScript (JSX), ESM ("type": "module")	—
Hosting	Cloudflare Workers (static SPA)	—
Live data	LeetCode via alfa-leetcode-api.onrender.com	runtime

---

2. Why each piece was chosen

Vite (build tool / dev server)

- Instant dev server with HMR and a near-zero-config React plugin

(`@vitejs/plugin-react`), so the inner loop is `npm install` → `npm run dev`.

- Production build (`vite build`) emits a fully static `dist/` (JS + CSS + assets)

with automatic code-splitting and hashing — ideal for dropping onto any static host. No SSR, no server runtime to maintain.

React 19

- Component model keeps the page declarative: one `App.jsx` holds all state

(`activeTab`, `leet`, `reduceMotion`) and renders every section.

- React 19's modern concurrent features and smaller runtime keep the bundle lean.
- No router needed — it's a single scrolling page with anchor (`#id`) navigation,

so React's core is all we require.

GSAP + ScrollTrigger

- GSAP is the animation workhorse: hero word reveals, a magnetic portrait that

eases toward the pointer, an orbital platform diagram, 3D tilt on project cards,
a scrubbed scroll-progress bar, and marquee motion.

- `ScrollTrigger` pins/syncs animations to scroll position (`scrub`) so reveals and

parallax feel physically tied to the page, not timer-based.

- `gsap.context()` + `ctx.revert()` scopes all animations to the root and tears them

down cleanly on unmount — important because the motion setup re-runs when the
reduced-motion preference changes.

Motion (Framer Motion) — only `useReducedMotion`

- Used purely as a tiny, reliable hook to detect the OS "reduce motion" setting.
- When reduced motion is on, the entire GSAP setup early-returns, so the site

degrades to a calm, static, readable page. This is an accessibility decision, not
decoration.

@phosphor-icons/react

- Tree-shakeable React icon components (GitHub, LinkedIn, envelope, list, seal-check,

arrow). Avoids shipping a whole icon font; only the imported glyphs are bundled.

Plain CSS with custom properties (not Tailwind / CSS-in-JS)

- One `styles.css` driven by a small set of design tokens (`:root` variables:

`--parchment`, `--ink`, `--ember`, `--bone`, serif font stacks, `--radius`).

- No build-time CSS framework means zero extra config, no purge rules, and the

exact editorial/parchment aesthetic (Iowan/Baskerville serif, ember-red accents)
is hand-controlled. The visual identity is the point, so bespoke CSS beats a
utility framework here.

ESLint 9 (+ react-hooks, react-refresh)

- Flat config (`eslint.config.js`) enforces recommended JS rules and the React Hooks

rules so effect/dependency mistakes surface before runtime. `no-unused-vars` is

relaxed and `react-refresh/only-export-components` is a warn so the single-file `App.jsx` (which exports both the component and module-level data arrays) doesn't spam errors.

---

3. Architecture & key features

Single file, `src/App.jsx` (~510 lines), structured as:

- **Module-level data constants** — `platforms`, `methods`, `marqueePhrases`, `tabs`, `ghRepos`, `LEET_FALLBACK`, `life`, `resume`, `skillGroups`. All content (projects, résumé, skill chips) lives here as plain data, so editing the site is editing arrays, not JSX.
- App **component** — holds UI state and runs three `useEffect` blocks:

1. Live LeetCode fetch + the full GSAP motion setup (guarded by reduced motion).
2. Tab-panel entrance animation.

- `src/main.jsx` — mounts `<App/>` inside `<StrictMode>` and imports `styles.css`.
- `index.html` — minimal shell with `#root` and the module script.
- `vite.config.js` — just the React plugin.

Sections (in scroll order)

1. **Nav** — wordmark + mobile menu link to `#presence`.
2. **Hero** — animated headline ("Data science / made legible"), magnetic portrait, and an interactive **orbital diagram** linking the five platforms (GitHub, Kaggle, PyPI, LeetCode, LinkedIn).
1. **Masthead** — scrubbed marquee of declaration phrases.
2. **Work** — "Off the clock" photo gallery (`/public/life/*`) with gentle, bounded auto-shuffle (paused under reduced motion).
1. **Method** — four-step working-method list, staggered in on scroll.
2. **Presence** — tabbed hub (GitHub repos, Kaggle dataset, PyPI package, live LeetCode stats with the crying banana-cat mascot, and an inline résumé).
1. **Tools / profile** — skill chips grouped by category, staggered on scroll.
2. **Footer** — contact CTA + profile links.

Interactive details

- **Custom cursor** — dot + lagging ring via `gsap.quickTo`; ring grows on

interactive elements.

- **Magnetic portrait** + parallax on scroll.
- **3D tilt** on project cards following the pointer.
- **Scroll progress bar** fixed to the top, scaled by scroll.
- **Reduced-motion aware** — all of the above is skipped when the OS requests it.

---

4. Live data & fallbacks

- **LeetCode stats**: fetched at runtime from

`https://alfa-leetcode-api.onrender.com/shivmahlan`. On success it shows solved count, acceptance rate, easy/medium/hard split, languages, top tags, global rank, and a "live" indicator dot.

- **Fallback**: if the API is unreachable, `LEET_FALLBACK` (résumé-derived numbers)

is shown instead — the UI never breaks, it just shows static best-known values.
The fetch is cancelled on unmount to avoid state updates after teardown.

- **Résumé download**: the PDF is embedded as a base64 data URI (`src/resume-base64.js` → `RESUME_B64`) and served via a `data:application/pdf;base64,...` download link.
This is deliberate: it sidesteps Cloudflare Workers occasionally 404-ing static file downloads, so the résumé always downloads.

- **LeetCode mascot**: an external Tenor GIF (crying banana-cat) — pure personality,

lazy-loaded, `aria-hidden`.

No API keys, no backend, no build-time secrets. Everything sensitive runs client-side or is public data.

---

5. Project layout

```
portfolio/
├── index.html              # SPA shell
├── package.json            # deps + scripts (dev/build/lint)
├── vite.config.js          # React plugin only
├── eslint.config.js        # flat ESLint config
├── public/
│   ├── resume.pdf          # source PDF (also embedded as base64)
│   ├── hero-portrait.*     # portrait image
│   └── life/               # personal gallery photos
└── src/
    ├── main.jsx            # React mount + StrictMode
    └── App.jsx             # entire app (state, data, animation, markup)
```

```
■■■ styles.css          # design tokens + all component styles
■■■ resume-base64.js    # base64 résumé for the download link
```

---

6. Commands

```
npm install        # install dependencies
npm run dev        # dev server (http://localhost:5173)
npm run build      # production build -> dist/
npm run preview    # preview the production build locally
npm run lint       # ESLint check
```

---

7. Deployment (Cloudflare Workers, static SPA)

- Framework preset: **Vite**.
- Build command: `npm run build`
- Output directory: `dist`
- No server or API keys required — profile data is pulled client-side at runtime.
- The résumé is embedded as base64 specifically so the download works even though

Workers can occasionally 404 static assets.

---

8. Trade-offs & notes

- **One big** `App.jsx` keeps the project approachable (edit data arrays, not JSX)

at the cost of a long file; fine for a single-author portfolio.

- **GSAP, not CSS-only animation**, buys scroll-scrubbed, physics-eased motion that

pure CSS can't express as cleanly.

- **Motion lib included only for** `useReducedMotion` — a small dependency kept

because it's the most robust cross-browser way to read the OS preference.

- **External GIF/API dependencies** mean two features (LeetCode mascot, live stats)

depend on third-party uptime; both have fallbacks or are non-critical.